
Bases de Datos - CE 3101

Documentación Técnica TecAir

Por

Bagliano Gomez Aldo Alessandro
Altamirano Cordero Esteban Andrés
Alfaro Mayorga Andrés Mauricio
Retana Murillo Braulio

Profesor: Marco Rivera Meneses

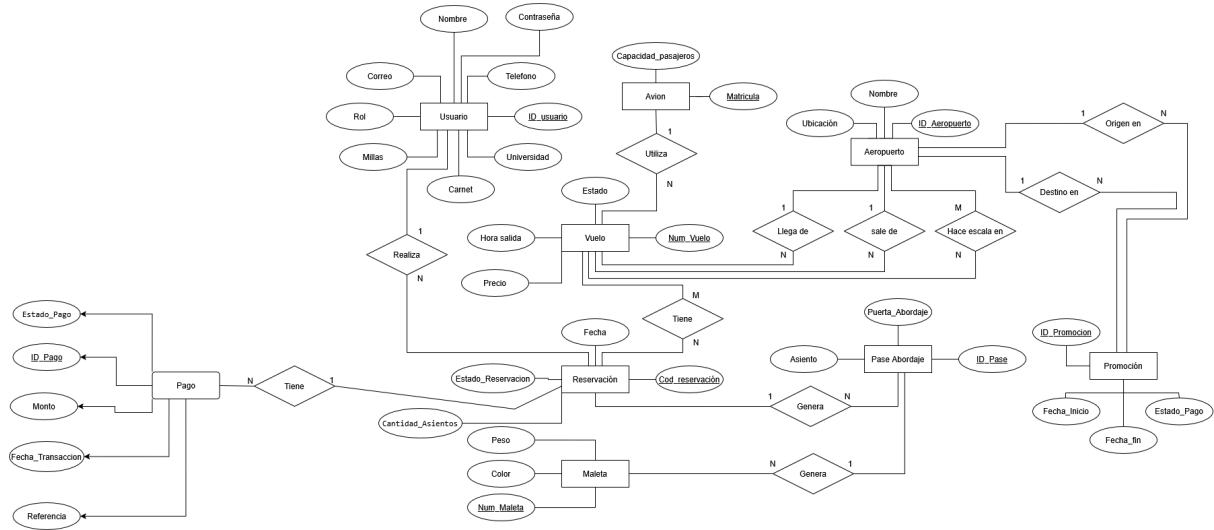
26/05/2026 23:59h

Índice

Índice.....	1
Modelo conceptual.....	2
Descripción Mapeo del modelo ER a Relacional.....	3
Diagrama Relacional.....	5
Descripción de la estructuras de datos desarrolladas (Relaciones).....	6
Descripción detallada de la arquitectura desarrollada.....	8
1. Capa de presentación.....	8
Web App – Vista Reservaciones.....	8
Web App – Vista Aeropuerto.....	8
Mobile App.....	9
2. Capa de sincronización y acceso a datos.....	9
SQLite.....	9
SyncProcess.....	9
3. Capa de servidor de aplicaciones.....	10
Application Server.....	10
4. Capa de API y servicios.....	10
REST API Service.....	10
5. Capa de persistencia de datos.....	11
DataBase Server.....	11
PostgreSQL.....	11
Problemas conocidos (que no tuvieron solución).....	13
Problema encontrados.....	14
Bibliografía consultada para el proyecto.....	19
Minutas.....	21

Modelo conceptual

A continuación se detalla el modelo conceptual o modelo entidad relación del proyecto



Descripción Mapeo del modelo ER a Relacional

En primer lugar, se realizó el mapeo de entidades fuertes. Para cada entidad fuerte presente en el diagrama ER se creó una tabla en el modelo relacional que contiene todos sus atributos simples junto con su respectiva llave primaria. De esta manera se obtuvieron las tablas USUARIO, AVION, AEROPUERTO, VUELO, RESERVACION, PASE_ABORDAJE, MALETA, PROMOCION y PAGO.

La entidad USUARIO se transformó en una tabla que almacena la información de cada usuario del sistema, incluyendo atributos como nombre, correo, teléfono, rol, millas, carnet, universidad y contraseña. La llave primaria utilizada fue ID_Usuario.

La entidad AVION se convirtió en una tabla que contiene la matrícula del avión, su capacidad de pasajeros y el número de asientos. En este caso, la matrícula funciona como llave primaria.

Para la entidad AEROPUERTO se creó una tabla con el identificador del aeropuerto, su nombre y ubicación, utilizando ID_Aeropuerto como llave primaria.

Posteriormente, la entidad VUELO fue transformada en una tabla que almacena la información relacionada con cada vuelo, como el número de vuelo, la hora de salida, hora de llegada y estado del vuelo. Además, esta tabla incluye varias llaves foráneas producto de las relaciones existentes en el modelo ER.

La entidad RESERVACION se mapeó como una tabla encargada de almacenar las reservaciones realizadas por los usuarios. Esta contiene información como la fecha y el estado de la reservación, además de las llaves foráneas que permiten relacionarla tanto con el usuario que realiza la reservación como con el vuelo reservado.

La entidad PASE_ABORDAJE se transformó en una tabla que almacena la información del pase generado para cada reservación, incluyendo atributos como asiento, puerta de abordaje y hora de impresión.

La entidad MALETA se convirtió en una tabla independiente donde se almacena la información de las maletas registradas, como el peso y color de cada una.

Asimismo, la entidad PROMOCION fue transformada en una tabla que contiene la información de promociones disponibles dentro del sistema, incluyendo fechas de inicio y finalización, estado de activación e información relacionada con aeropuertos de origen y destino.

Finalmente, la entidad PAGO se mapeó como una tabla encargada de almacenar los pagos asociados a las reservaciones, incluyendo monto, fecha de transacción, referencia y estado del pago.

Después del mapeo de entidades fuertes, se analizó el uso de entidades débiles. Sin embargo, dentro del modelo conceptual no se identificaron entidades débiles, por lo que este paso no aplicó durante el proceso de transformación.

Posteriormente, se realizó el análisis de relaciones binarias 1:1. En el modelo ER no se identificaron relaciones con cardinalidad uno a uno, por lo que este tipo de mapeo tampoco fue necesario.

Luego se procedió al mapeo de relaciones binarias 1:N. Para estas relaciones se siguió la regla general de agregar la llave primaria de la entidad del lado “1” como llave foránea dentro de la tabla ubicada en el lado “N”.

La relación entre USUARIO y RESERVACION se implementó agregando la llave foránea ID_Usuario dentro de la tabla RESERVACION, permitiendo identificar qué usuario realizó cada reservación.

La relación entre AVION y VUELO se implementó agregando la matrícula del avión como llave foránea dentro de la tabla VUELO, asociando cada vuelo con el avión que lo realiza.

Las relaciones entre AEROPUERTO y VUELO también fueron mapeadas mediante llaves foráneas. En este caso se agregaron los atributos ID_Aeropuerto_Origen e ID_Aeropuerto_Destino dentro de la tabla VUELO para representar los aeropuertos de salida y llegada de cada vuelo.

La relación entre RESERVACION y PASE_ABORDAJE se implementó agregando el código de reservación como llave foránea en la tabla PASE_ABORDAJE, permitiendo asociar cada pase de abordaje con una reservación específica.

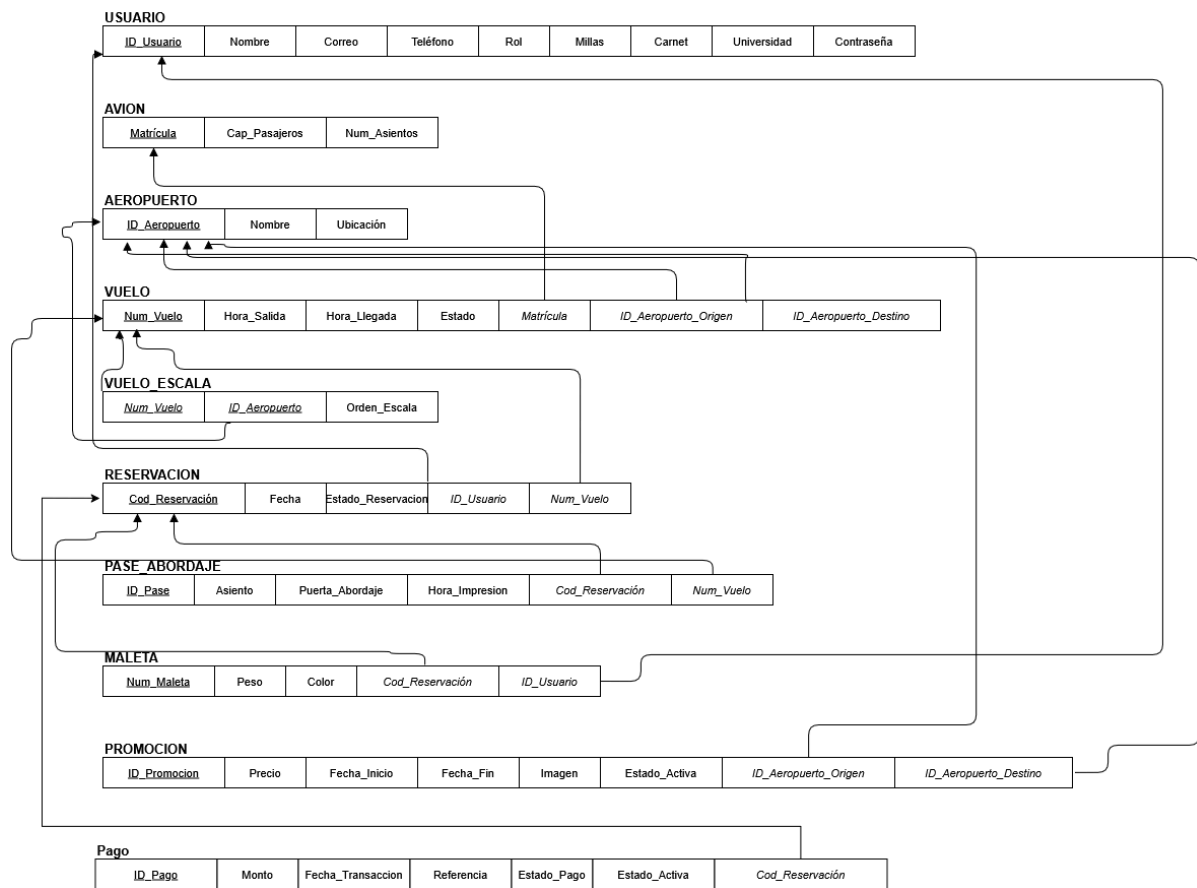
De igual manera, la relación entre RESERVACION y MALETA se representó mediante la inclusión de Cod_Reservacion como llave foránea dentro de la tabla MALETA.

Asimismo, la relación entre RESERVACION y PAGO se implementó agregando el código de reservación en la tabla PAGO, permitiendo relacionar múltiples pagos con una misma reservación.

Posteriormente se realizó el mapeo de relaciones binarias N:M. En el modelo ER se identificó una relación de muchos a muchos entre VUELO y AEROPUERTO correspondiente a las escalas realizadas por los vuelos. Para representar esta relación se creó una tabla intermedia llamada VUELO_ESCALA. Esta tabla contiene como llaves foráneas el número de vuelo y el identificador del aeropuerto, formando conjuntamente una llave primaria compuesta. Además, se incluyó el atributo Orden_Escala, el cual representa el orden en que se realiza cada escala dentro del vuelo.

Finalmente, se revisó el uso de atributos multivaluados dentro del modelo conceptual. Debido a que no se identificaron atributos de este tipo, este paso del proceso de mapeo no fue necesario.

Diagrama Relacional



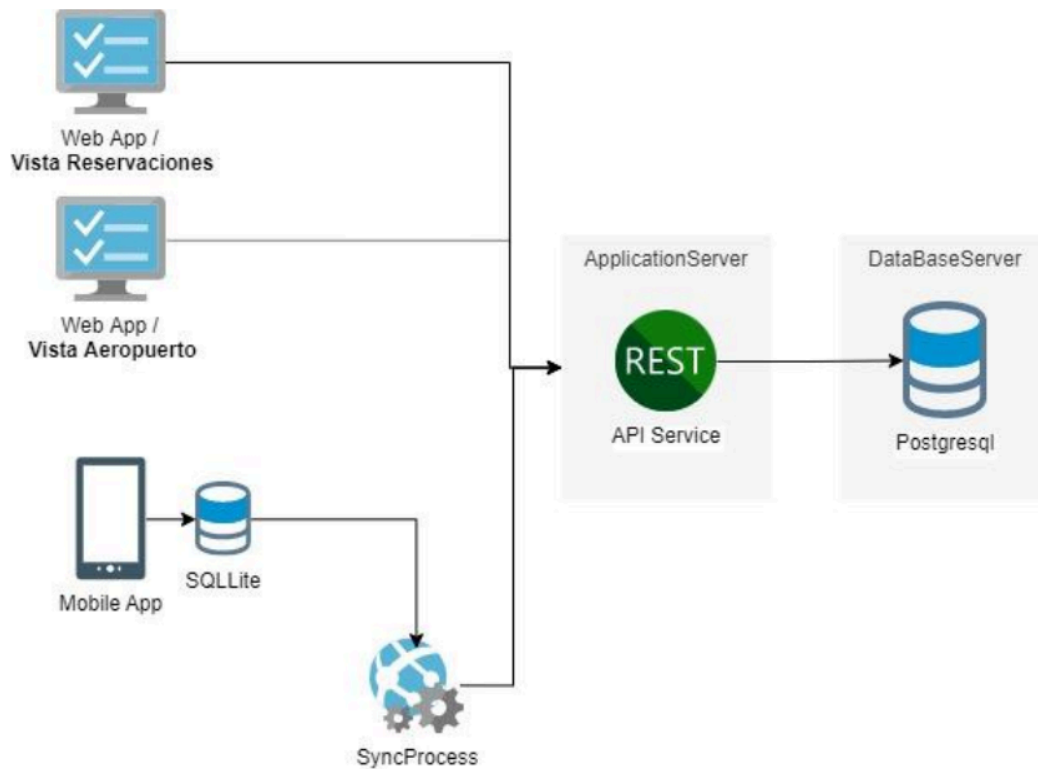
Descripción de la estructuras de datos desarrolladas (Relaciones)

Estructura de Datos (Relación)	Descripción
Usuario – Reserva	Un usuario puede realizar múltiples reservas dentro del sistema. Esta relación permite asociar cada reserva con el cliente que la realizó, facilitando el historial de compras y la administración de vuelos reservados.
Reserva – Vuelo	Cada reserva puede estar asociada a uno o varios vuelos, permitiendo gestionar itinerarios y controlar la disponibilidad de espacios en cada vuelo.
Vuelo – Ruta	Cada vuelo pertenece a una ruta específica. Esta relación permite definir el trayecto que seguirá el vuelo, incluyendo aeropuerto de origen y destino.
Ruta – Aeropuerto	Una ruta se compone de aeropuertos de salida y llegada. Esta relación permite estructurar correctamente los trayectos operados por la aerolínea.
Vuelo – Avión	Cada vuelo utiliza un avión determinado. Esto permite administrar la disponibilidad de aeronaves y asociar características específicas como capacidad y modelo.
Avión – Asiento	Un avión contiene múltiples asientos. Esta relación permite gestionar la distribución interna de pasajeros y la asignación de espacios durante las reservas.
Reserva – Asiento	Una reserva puede incluir la asignación de uno o varios asientos. Esto facilita controlar cuáles espacios ya fueron ocupados por pasajeros.
Reserva – Equipaje	Cada reserva puede registrar equipaje asociado al pasajero. La relación permite controlar cantidad, peso y estado del equipaje registrado.
Reserva – Check-In	Una reserva puede generar un proceso de check-in previo al vuelo. Esta relación permite validar pasajeros y habilitar el abordaje.
Check-In – Boarding Pass	Después del check-in se genera un boarding pass o pase de abordar. Esta relación permite emitir el documento necesario para el ingreso al vuelo.
Vuelo – Promoción	Algunos vuelos pueden tener promociones asociadas. Esta relación permite aplicar

	descuentos o beneficios especiales a determinadas rutas o temporadas.
Usuario – Check-In	Un usuario puede realizar check-in para sus vuelos reservados. Esto permite mantener trazabilidad de los pasajeros que confirmaron su asistencia.
Usuario – Equipaje	El sistema relaciona usuarios con su equipaje registrado, permitiendo identificar pertenencias y mantener control logístico.
Aeropuerto – Vuelo	Un aeropuerto puede participar en múltiples vuelos tanto de salida como de llegada. Esta relación permite gestionar operaciones aeroportuarias dentro del sistema.
Avión – Vuelo	Un avión puede ser utilizado en distintos vuelos en diferentes horarios. Esto permite optimizar la planificación y administración de aeronaves.

Descripción detallada de la arquitectura desarrollada

A continuación se detalla la arquitectura desarrollada en el proyecto y la descripción de esta



1. Capa de presentación

La capa de presentación corresponde a las interfaces utilizadas por los diferentes tipos de usuarios del sistema. Esta capa fue desarrollada con el objetivo de proporcionar acceso a las funcionalidades de la plataforma de manera intuitiva y organizada.

Web App – Vista Reservas

Esta aplicación web está enfocada en la gestión de reservas realizadas por los usuarios del sistema. Desde esta interfaz es posible consultar vuelos disponibles, realizar reservas, seleccionar asientos, gestionar información del pasajero y consultar el estado de las reservas.

La aplicación consume los servicios expuestos por la REST API mediante solicitudes HTTP, permitiendo una comunicación desacoplada con el servidor de aplicaciones.

Web App – Vista Aeropuerto

Corresponde a una segunda interfaz web orientada a operaciones administrativas y de aeropuerto. Esta vista permite gestionar procesos asociados a vuelos, check-in, abordaje, equipaje y monitoreo de operaciones aeroportuarias.

La separación entre ambas aplicaciones web permite manejar distintos perfiles de usuario y responsabilidades dentro del sistema.

Mobile App

La aplicación móvil fue desarrollada para permitir operaciones desde dispositivos móviles, especialmente en escenarios donde puede existir conectividad limitada o intermitente.

La aplicación tiene integración con una base de datos local SQLite, permitiendo almacenar información temporalmente y operar parcialmente en modo offline. Posteriormente, los cambios realizados se sincronizan con el servidor central mediante el proceso de sincronización implementado.

2. Capa de sincronización y acceso a datos

Esta capa tiene la responsabilidad de administrar el almacenamiento local en la aplicación móvil y garantizar la consistencia de la información entre los dispositivos móviles y la base de datos central del sistema.

SQLite

SQLite funciona como una base de datos ligera embebida dentro de la aplicación móvil. Su principal objetivo es permitir que ciertas operaciones puedan ejecutarse aun cuando no exista conexión directa con el servidor principal.

Entre los datos almacenados localmente se incluyen registros necesarios para operaciones temporales, consultas rápidas y manejo de información pendiente de sincronización.

SyncProcess

El módulo de sincronización es responsable de transferir y actualizar los datos entre SQLite y PostgreSQL.

Este proceso permite:

- Detectar cambios realizados localmente.
- Enviar actualizaciones al servidor central.
- Descargar información nueva desde el servidor.
- Mantener consistencia entre dispositivos.
- Gestionar posibles conflictos de sincronización.

La sincronización se realiza mediante comunicación con la REST API del sistema, utilizando solicitudes HTTP para el intercambio de información.

3. Capa de servidor de aplicaciones

Application Server

El servidor de aplicaciones concentra toda la lógica de negocio del sistema. Esta capa fue implementada utilizando una arquitectura por capas en .NET, separando adecuadamente controladores, servicios, repositorios y modelos de datos.

Dentro de esta capa se implementan funcionalidades relacionadas con:

- Gestión de vuelos.
- Reservaciones.
- Usuarios.
- Check-in.
- Equipaje.
- Asignación de asientos.
- Promociones.
- Boarding pass.

La lógica de negocio se encuentra desacoplada de la capa de presentación y de la capa de persistencia, permitiendo una arquitectura más mantenible y escalable.

Además, el servidor utiliza patrones de diseño como:

- Repository Pattern.
- DTOs (Data Transfer Objects).
- Separación por servicios e interfaces.

Esto facilita la reutilización de código, las pruebas unitarias y la mantenibilidad general del proyecto.

4. Capa de API y servicios

REST API Service

La REST API funciona como intermediaria entre los clientes y el servidor de aplicaciones. Su responsabilidad es exponer los servicios del sistema mediante endpoints accesibles a través del protocolo HTTP.

Las aplicaciones web y móvil consumen esta API para realizar operaciones como:

- Consultar vuelos.
- Crear reservaciones.
- Realizar check-in.

- Gestionar usuarios.
- Consultar promociones.
- Sincronizar datos móviles.

La API fue diseñada bajo principios REST, permitiendo una comunicación desacoplada y facilitando la integración entre diferentes plataformas cliente.

Además, el uso de una API centralizada mejora la escalabilidad y simplifica el mantenimiento del sistema, ya que toda la lógica de acceso a funcionalidades se encuentra concentrada en un único punto de entrada.

5. Capa de persistencia de datos

DataBase Server

El servidor de base de datos almacena toda la información centralizada del sistema y representa la fuente principal de datos de la plataforma.

Esta capa garantiza persistencia, integridad y consistencia de la información utilizada por todas las aplicaciones del ecosistema TECAir.

PostgreSQL

Como motor de base de datos relacional se utilizó PostgreSQL, debido a sus capacidades de escalabilidad, confiabilidad y soporte para relaciones complejas.

En PostgreSQL se almacena información relacionada con:

- Usuarios.
- Reservaciones.
- Vuelos.
- Rutas.
- Aeropuertos.
- Aviones.
- Equipaje.
- Asientos.
- Check-in.
- Promociones.

El flujo principal del sistema inicia cuando un usuario interactúa con alguna de las aplicaciones cliente (web o móvil). Estas aplicaciones realizan solicitudes a la REST API, la cual redirige las operaciones al servidor de aplicaciones.

El servidor procesa la lógica de negocio correspondiente y posteriormente realiza operaciones sobre la base de datos PostgreSQL mediante la capa de acceso a datos.

Problema encontrados

Problema	Descripción	Solución
Inconsistencia en esquema relacional de PostgreSQL (columna password_hash faltante)	Al intentar iniciar sesión o registrar clientes en la plataforma, las peticiones HTTP retornaban errores 500 Internal Server Error. En los logs de la API desarrollada en C# se identificó la excepción Npgsql.PostgresException: 42703: no existe la columna «password_hash». Esto impedía completar correctamente los procesos de autenticación y registro de usuarios. Inicialmente se contempló omitir temporalmente el manejo de contraseñas cifradas, pero esto incumplía los requerimientos de seguridad establecidos para el proyecto.	Se localizó y ejecutó correctamente el script Migration_AddPasswordHash.sql utilizando psql.exe. La migración agregó la columna faltante y habilitó la extensión criptográfica pgcrypto de PostgreSQL, permitiendo utilizar funciones como gen_salt('bf') y crypt para el manejo seguro de contraseñas mediante BCrypt.
Incompatibilidad del DatePicker en plataformas híbridas (Expo Web vs dispositivos móviles)	El selector de fechas funcionaba correctamente en dispositivos móviles Android e iOS, pero generaba errores críticos al ejecutarse en navegadores web mediante Expo Web. El error principal indicaba incompatibilidad de propiedades de estilo CSS con el componente @react-native-community/date-timepicker. Esto ocasionaba fallos de compilación y bloqueos del bundle de TypeScript. Los intentos iniciales de modificar directamente los estilos del componente no resolvían el problema y generaban nuevas incompatibilidades entre plataformas.	Se implementó una lógica de renderizado condicional utilizando Platform.OS === 'web'. Para navegadores web se sustituyó el componente nativo por un control HTML5 <input type="date"> con estilos compatibles y manejo seguro de zona horaria. El componente DateTimePicker se mantuvo únicamente para dispositivos móviles físicos ejecutados mediante Expo Go.
Error de carga de vuelos por columna imprecisa (f.price faltante)	Las consultas de vuelos realizadas desde la aplicación web y móvil producían pantallas de carga infinitas y errores inesperados. El problema se originaba porque	Se aplicó la migración Migration_AddFlightPrice.sql sobre la base de datos PostgreSQL, agregando correctamente la columna price DECIMAL(10,2) junto con

	el repositorio FlightRepository.cs intentaba consultar la columna f.price en la tabla flights, pero dicha columna no existía dentro del esquema original de PostgreSQL. Inicialmente se intentó simular el precio directamente desde la lógica de C#, pero esto comprometía la integridad y persistencia de los datos.	restricciones CHECK (price >= 0). Posteriormente las búsquedas y consultas de vuelos comenzaron a funcionar correctamente tanto en la aplicación web como móvil.
Bloqueo de red por Firewall de Windows y direcciones IP incorrectas en Expo Metro	La aplicación móvil no lograba conectarse a la API REST local debido a que localhost dentro del dispositivo móvil hacía referencia al propio teléfono y no a la computadora de desarrollo. Esto producía errores del tipo Axios Network Error. Adicionalmente, el Firewall de Windows bloqueaba las conexiones externas hacia el puerto utilizado por la API. Se intentó utilizar Ngrok como solución alternativa, pero sin una configuración completa del servicio se generaban errores adicionales relacionados con Metro Bundler.	Se creó una regla de entrada en el Firewall de Windows para permitir conexiones TCP hacia el puerto 5102 utilizado por la API. Además, se configuró la variable EXPO_PUBLIC_API_URL dentro del archivo .env apuntando hacia la dirección IP local de la red Wi-Fi (http://192.168.88.3:5102/api). Esto permitió que dispositivos móviles físicos pudieran consumir correctamente la API REST y sincronizar información entre la aplicación móvil y PostgreSQL.

Conclusiones del proyecto

Desde el punto de vista de bases de datos, el proyecto permitió aplicar conceptos de modelado entidad-relación, integridad referencial y manejo de relaciones complejas entre entidades. La estructura diseñada en PostgreSQL permitió representar de manera consistente los diferentes procesos de una aerolínea, tales como reservaciones, vuelos, check-in, equipaje y asignación de asientos.

El uso de tecnologías como .NET, React, React Native, PostgreSQL y SQLite permitió desarrollar una plataforma robusta y adaptable a distintos escenarios de operación. La integración entre frontend, backend y base de datos demostró la importancia de una correcta comunicación entre capas mediante servicios REST.

El uso de github o cualquier otra herramienta de gestión de código es imprescindible para un proyecto de este tipo , ya que permite una organización adecuada para trabajar entre las diferentes capas , la estructura de ramas que se trabajó en el proyecto fue de gran ayuda para evitar problemas a la hora de hacer merge en las ramas ya que el código estaba bien organizado.

El proyecto permitió poner en práctica buenas prácticas de ingeniería de software, como el uso de DTOs, patrones Repository, separación de servicios e implementación de pruebas. Esto contribuyó a obtener un sistema más organizado, mantenible

Recomendaciones del proyecto.

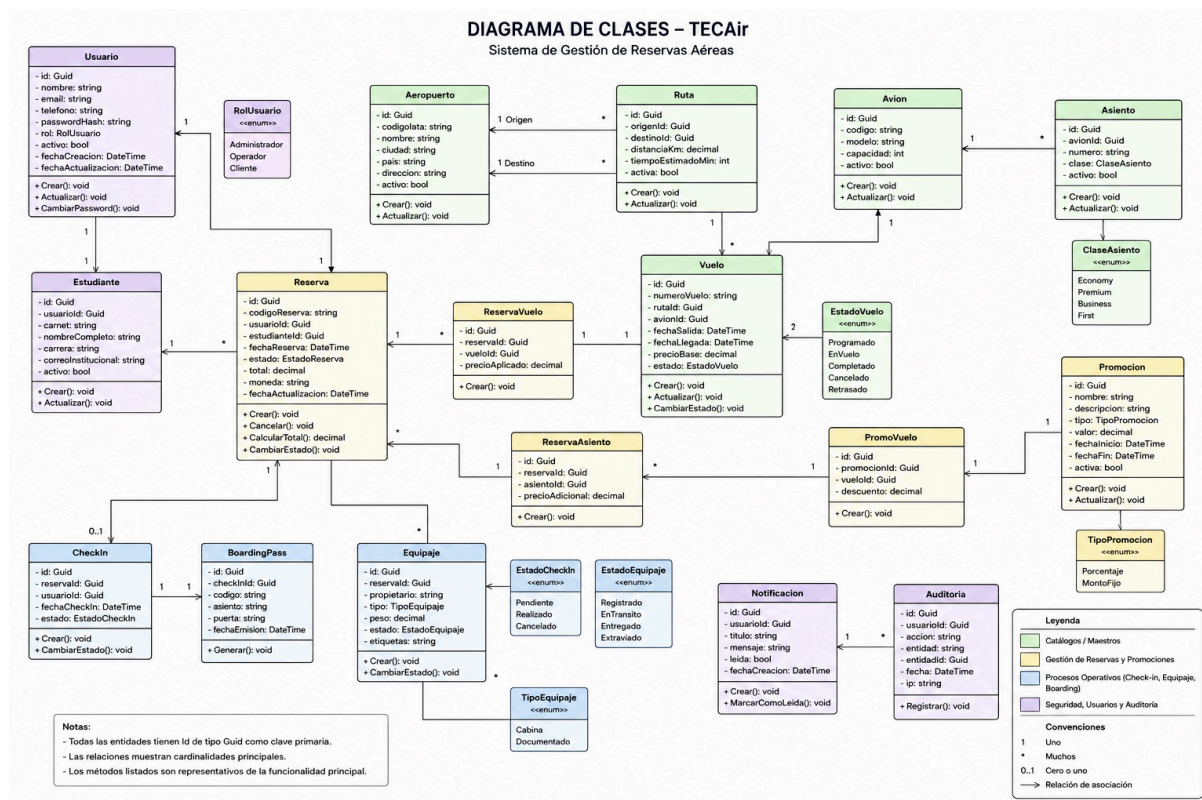
Uno de los aspectos que presentó mayores retos durante el desarrollo fue la aplicación móvil, especialmente en temas relacionados con configuración del entorno, compatibilidad entre dependencias, sincronización de estados y manejo de compilaciones. Debido a esto, se recomienda que se utilice Android Studio como entorno principal para el desarrollo y pruebas móviles, ya que ofrece herramientas más completas para depuración, emulación de dispositivos y monitoreo del rendimiento de la aplicación.

Se recomienda mejorar la estructura interna del código móvil mediante una organización más modular de componentes, pantallas y servicios.

También se recomienda implementar una mejor gestión de estados dentro de la aplicación móvil, utilizando herramientas o arquitecturas más robustas para manejar información compartida entre pantallas y procesos de sincronización. Esto ayudaría a reducir errores relacionados con actualización de datos y comportamiento inconsistente de la interfaz.

Desde el punto de vista de rendimiento, sería beneficioso optimizar las cargas de información y el renderizado de componentes móviles para evitar lentitud en ciertas operaciones o pantallas con gran cantidad de datos.

Diagrama de clases y un documento que explique el porqué del diseño.



Descripción del por qué del diseño

La decisión de utilizar una arquitectura multicapa surge de la necesidad de desacoplar la interfaz de usuario, la lógica de negocio y el acceso a datos. Esto permitió que cada componente del sistema pudiera evolucionar de manera independiente sin afectar directamente las demás capas. Gracias a esta separación, fue posible desarrollar simultáneamente la aplicación web, la aplicación móvil y los servicios backend.

El diseño del backend se realizó utilizando una arquitectura organizada en capas como API, Core y Data. Esta distribución permitió centralizar la lógica de negocio en servicios especializados y mantener el acceso a datos desacoplado mediante el uso de repositorios e interfaces. Además, la utilización de DTOs ayudó a controlar el intercambio de información entre capas y clientes externos, mejorando la seguridad y la mantenibilidad del sistema.

En cuanto a la base de datos, se optó por PostgreSQL debido a su robustez, estabilidad y capacidad para manejar relaciones complejas entre entidades. El modelo de datos fue diseñado siguiendo principios de normalización e integridad referencial, permitiendo representar adecuadamente procesos reales de una aerolínea, como reservaciones, vuelos, check-in, equipaje y asignación de asientos.

La estructura relacional implementada buscó minimizar redundancia de información y mantener consistencia entre las entidades del sistema mediante el uso de llaves primarias y foráneas. Además, el diseño permitió manejar relaciones uno a muchos y muchos a muchos necesarias para representar correctamente el dominio del problema.

Para la capa de presentación web se utilizaron tecnologías basadas en React, permitiendo desarrollar interfaces dinámicas y reutilizables. La separación de vistas según el tipo de usuario permitió diferenciar claramente las funcionalidades orientadas a clientes y aquellas dirigidas al personal operativo del aeropuerto.

En el caso de la aplicación móvil, el diseño se enfocó en brindar portabilidad y capacidad de operación en distintos escenarios. Debido a que uno de los requerimientos del proyecto contemplaba sincronización de información y soporte parcial offline, se incorporó un mecanismo de almacenamiento local y sincronización con el servidor central.

Durante el desarrollo móvil se identificó que la gestión de estados, compatibilidad de dependencias y manejo de sincronización representaban algunos de los retos más complejos del proyecto. Esto influyó en la forma en que se estructuraron las pantallas, servicios y módulos internos de la aplicación

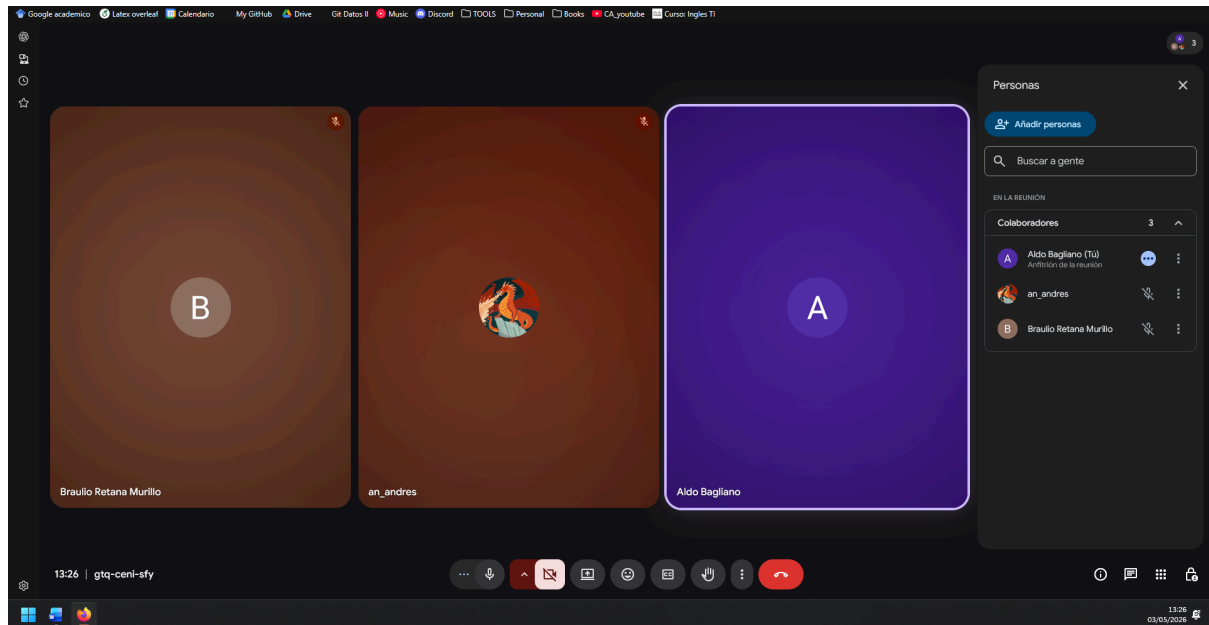
Bibliografía consultada para el proyecto

- Microsoft. (2024). ASP.NET Core documentation. <https://learn.microsoft.com/en-us/aspnet/core/>
- Microsoft. (2024). C# Programming Guide. <https://learn.microsoft.com/en-us/dotnet/csharp/>
- Npgsql. (2024). Npgsql — .NET Data Provider for PostgreSQL. <https://www.npgsql.org/doc/>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- Meta Open Source. (2024). React Documentation. <https://react.dev/>
- Expo. (2024). Expo Documentation. <https://docs.expo.dev/>
- Expo. (2024). expo-sqlite — SQLite local database. <https://docs.expo.dev/versions/latest/sdk/sqlite/>
- React Bootstrap. (2024). React-Bootstrap Documentation. <https://react-bootstrap.netlify.app/>
- Vite. (2024). Vite — Next Generation Frontend Tooling. <https://vitejs.dev/>
- BCrypt.Net. (2024). BCrypt.Net-Next NuGet Package. <https://github.com/BcryptNet/bcrypt.net>
- Axios. (2024). Axios HTTP Client. <https://axios-http.com/docs/intro>
- AngularJS. (2022). Angular Documentation. Recuperado de: <https://angular.io/>
- Bootstrap. (2022). Bootstrap Themes & Templates. Recuperado de: <https://wrapbootstrap.com/>

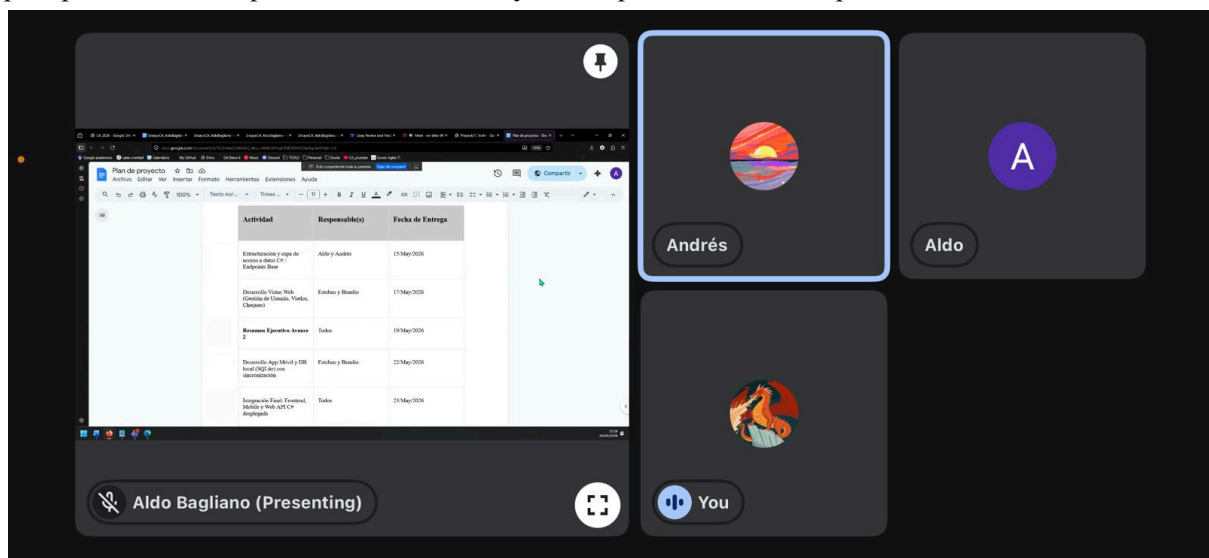
- Oracle. (2022). How to Write Doc Comments for the Javadoc Tool. Recuperado de:
<http://www.oracle.com/technetwork/articles/java/index-137868.html>
- Microsoft. (2022). C# Coding Conventions (C# Programming Guide). Recuperado de:
<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/inside-a-program/coding-conventions>

Minutas

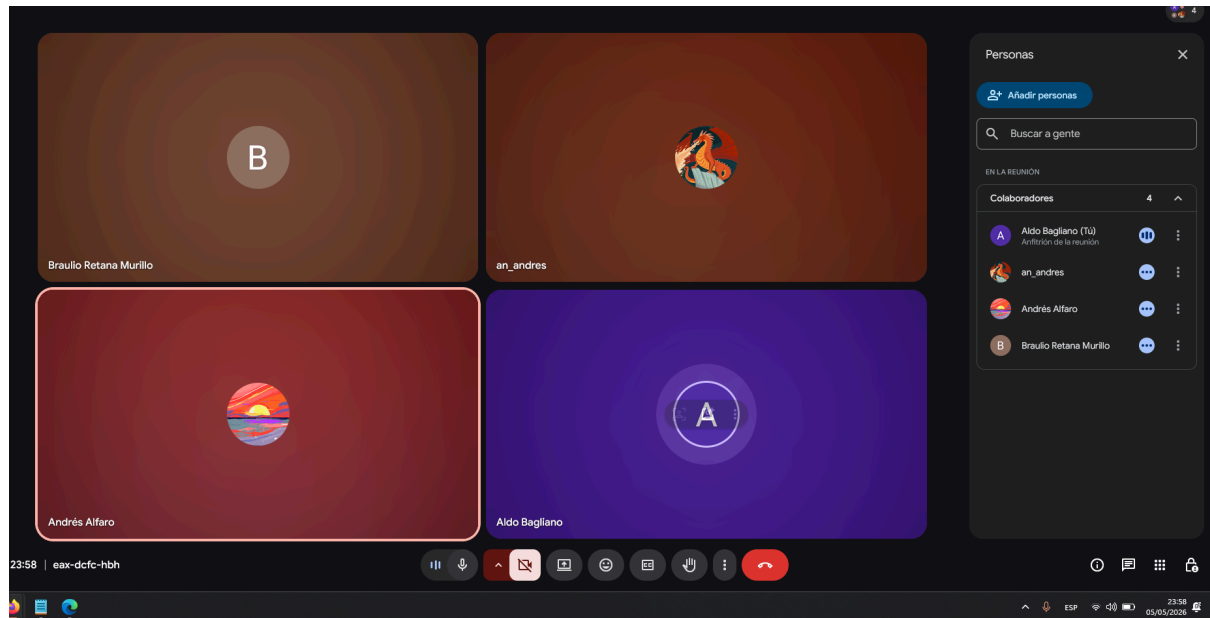
En primera instancia se hizo la primera reunión el 03/05/2026 para poder hacer todo el planteamiento del plan de proyecto, de esta manera se definió la forma en la que el grupo va a trabajar así como los responsables de cada tarea y las reglas que van a servir para realizar el proyecto de la mejor forma posible.



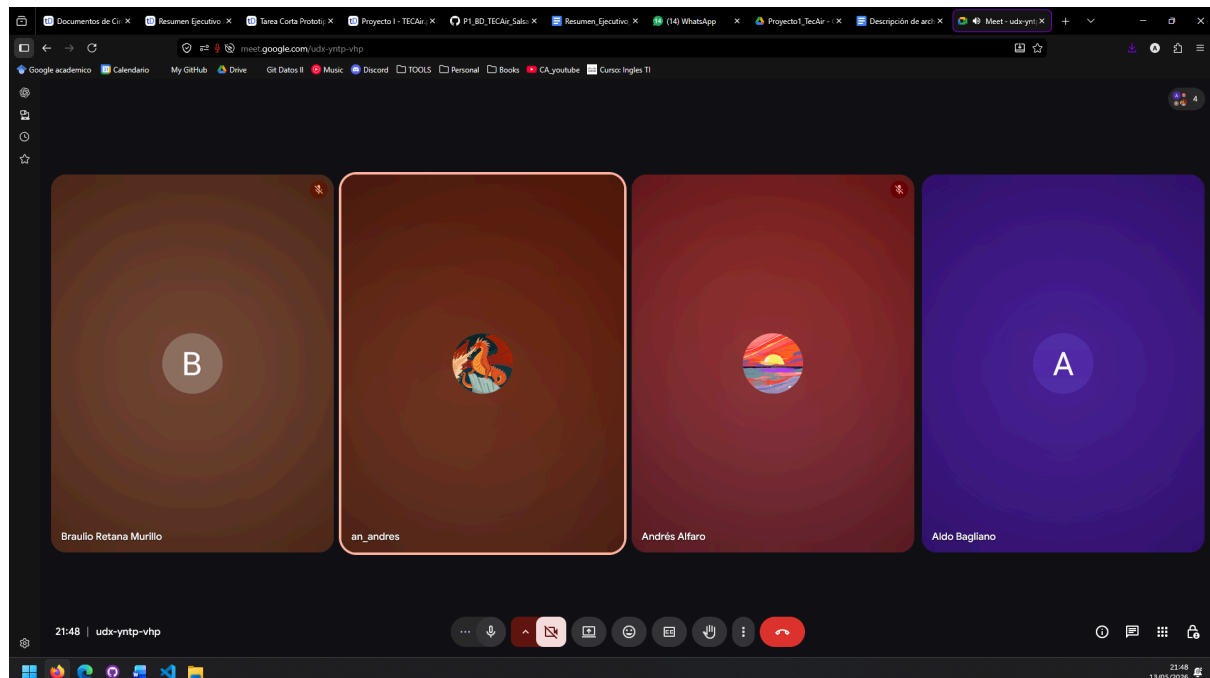
Seguidamente el día 04/05/2026 se procedió a hacer una revisión general del presente documento para poder verificar que todo esté en orden y se cumpla con todos los apartados del documento



El día 05/05/2026 se realizó una llamada para analizar el enunciado del proyecto e ir revisando el diagrama ER inicial del proyecto discutirlo para poder entregar una versión que cubra bien todos los requerimientos del proyecto, también se habló de la importancia de tener el repositorio de github bien organizado para evitar los conflictos con el código , de esto se añade que se van a trabajar 4 ramas principales y que de esas ramas se crean otras para trabajar cualquier funcionalidad nueva , cuando dicha funcionalidad se termine se le hace merge a la rama madre .



Reunión para revisar la estructura general del Backend y definir tecnologías para la sincronización de las BD y controladores del proyecto



Sesión de trabajo para poder unir el frontend con el backend se realizó el día 25/05/2026

The image is a composite of three main visual elements. On the left, a screenshot of the Visual Studio Code editor shows a Vue.js project. The main editor window displays a file named 'index.html' with a single-page application structure. It includes a header with a logo, a main content area with a 'Reservar' button, and a footer. The code uses Vue.js components and data binding. The right sidebar shows the file explorer with a tree view of the project files. Below the editor, a Windows taskbar is visible, showing the time as 20:50, the date as ddr-bdl-19-hh, and several open applications. On the right side of the image, there is a vertical strip showing three mobile app preview cards. The top card features a sunset logo and the name 'Andrés Alfaro'. The middle card shows a mobile app interface with a search bar and a list of items, with the name 'Braulio Retana Muriilo' below it. The bottom card is a solid purple color with a large white letter 'A' in the center and the name 'Aldo Bagliano' at the bottom.